115-1 NTU Challengers #PBL #AI Agent

【個別型提案】申請計畫書

一、基本資料

※備註:表格內空白處請確實填寫，若學生成員僅有1人，則成員2-5之相關資料皆無須填寫。

專案主題名稱:	從 RTL 到易讀硬體架構圖的自動化		
指導老師姓名	指導老師服務單位		指導老師聯絡信箱
楊奕軒	電信工程學研究所		yhyangtw@ntu.edu.tw
學生姓名	學生學號	學生系級	學生聯絡信箱
陳宇杰	b12901162	電機四	ajchen0508@gmail.com b12901162@ntu.edu.tw
薛晏閎	b12901064	電機四	b12901064@gmail.com
陳昱綸	b12901107	電機四	b12901107@gmail.com
成員4			
成員5			

二、專案內容說明

(一)專案目標

1. 團隊關注何種基於真實複雜現象產生的問題？

本團隊所關注的問題為:**在數位 IC 設計中，如何透過 AI 技術，從硬體程式碼中自動生成正確、且具備適當抽象層級(去蕪存菁)的硬體架構圖、狀態機與時序圖？**

在數位 IC 設計領域，硬體描述語言(HDL)是工程師開發硬體的基石，而暫存器傳輸級(RTL)則是撰寫此語言的核心抽象層級。要了解一顆 IC 如何運作或進行開發，有三種圖表至關重要:**硬體架構圖(Hardware Architecture)、狀態機(Finite State Machine)與時序圖(Timing Diagram)**。

本團隊成員皆具備電機系背景並修習過數位 IC 相關課程。我們在實際開發專案時深切體會到，若要從零打造完整的數位硬體架構，必須先定義好接腳、功能與通訊協定，而繪製上述三種圖表是最關鍵卻也最耗時的步驟。開發者必須仰賴這些圖表，才能快速理解系統並撰寫出正確的程式碼;反之，當工程師或學生需要參考他人的專案時，若缺乏標示清楚的圖表文件，僅憑閱讀龐雜的原始碼，幾乎無法快速掌握整體的設計邏輯。

此問題的真實性在於，這是所有硬體工程師與學生的日常痛點。不論是撰寫硬體規格文件、工程師之間的技術討論、接手理解他人的開發邏輯，甚至是學生的期末簡報，都需要有正確且簡明扼要的三大圖表，讓讀者或聽眾能在第一時間快速掌握整體架構，而不是一開始就被複雜且龐大的實體線路或訊號線給搞混。

此問題的複雜性在於，現有工具難以在「絕對正確」與「適度抽象」之間取得甜蜜點。

以傳統商用或開源軟體為例(如 Verdi 等 EDA 工具)：雖然能輸入程式碼並輸出 100% 正確的圖表，但當硬體系統層級過大時(例如:輸入一個完整系統單晶片 SoC 的模組)，軟體會將所有的微觀訊號線與底層邏輯閘全部展開。這會產生一張極度巨大且雜亂的架構圖，讀者根本無法從中看出資料流與控制單元的架構重點。

以目前的大語言模型(LLM)為例： 雖然可以直接輸入程式碼並要求其輸出標示重點的文字圖表(如 Mermaid 或 PlantUML 語法)，但其表現極不穩定。例如，模型可能會在生成時序圖時遺漏關鍵的握手訊號，或在硬體架構圖中錯置模組間的連線關係，甚至產生圖表排版歪斜、重點標錯等幻覺。

要界定哪部分該保留、哪部分該省略，本身就是一項複雜的認知工作。因此，如何透過 AI 的能力進行快速生成，並允許使用者要求不同的複雜度在「確保核心邏輯正確」的前提下，「將複雜度降低到只保留重點」正是本團隊致力解決的複雜挑戰。

2. 上述問題的重要性或是解決必要性為何？

本問題之所以重要，在於規格表中的硬體架構圖、狀態機圖與時序圖，在專案紀錄與知識傳遞上扮演著無可取代的角色。若缺乏有效率的製圖工具，將大幅拖慢研發節奏並增加溝通成本。

本問題解決之必要性，具體可以分為以下三個層次說明：對開發速度、對易理解性、以及對維護成本與文件一致性。

首先，對硬體開發速度而言：在現代 IC 設計中，開發速度往往是搶佔市場的關鍵，幾個月的時間就必須完成新一輪版本的迭代。然而，對於工程師而言，文件處理與製圖往往是最枯燥繁瑣，卻又不得不花費大量時間確保正確性的工作。在每一個專案中，文件規格與圖表的撰寫時間，通常會比實際撰寫程式碼的時間還要長，且單一專案需要製作的圖表可能高達上百張。目前使用現有繪圖軟體手動拉線製圖所花費的時間，甚至不亞於文字敘述。若能有一套好用的自動化且可信賴的 AI 製圖軟體，能直接讀懂程式碼並精準製圖，將為硬體工程師省下可觀的開發時間。

其次，對系統易理解性與團隊協作而言：大型數位 IC 專案通常是高度分工的，涵蓋了前端設計、驗證、後端合成等不同團隊。當一段程式碼需要交接給其他模組的負責人，或是當驗證工程師需要撰寫 Testbench 時，原始碼本身是難以快速消化的。如果能自動生成具備「適當抽象層級」的硬體架構圖與時序圖，就能為不同團隊建立一個直觀的「共同視覺語言」。這不僅能大幅降低新進人員的學習曲線，更能減少因程式碼理解錯誤而導致的系統整合失敗與 Bug。

最後，對降低維護成本與文件一致性而言： 硬體專案的生命週期通常很長，在迭代與除錯的過程中，工程師會頻繁修改 RTL 程式碼。然而，在趕進度的壓力下，開發者往往會忘記同步更新規格書中的圖表，導致「程式碼」與「設計文件」脫節，這在業界被稱為危險的技術債。若能開發出透過程式碼一鍵生成重點圖表的工具，就能確保文件永遠與最新版的程式碼保持絕對地一致性，在修改 RTL 程式碼的同時便自動更新規格表中的圖片。這不僅能確保設計規格的單一來源，更能大幅降低專案在未來數年內的維護與交接成本。

3. 此專案之執行預計涉及那些重要利害關係人（stakeholder）？

※備註：所謂「利害關係人」的常見定義，為任何有可能影響專案目標達成與否，或因專案目標達成而受影響的群體／個人。

種類	對象	影響
核心使用者	數位 IC 設計工程師 、硬體驗證工程師	改變規格文件的處理流程，免去手動繪製複雜硬體圖表的時間。精確的圖表將作為開發與驗證團隊的共同視覺語言，大幅降低交接溝通成本與驗證漏洞。
管理與決策者	研發主管（RD Manager）、專案經理（PM）	直接協助團隊縮短專案開發與交接週期。一鍵生成圖表能確保「程式碼」與「設計文件」保持強一致性，有效降低團隊技術債與專案長期維護成本。
學術教育界	電機／電子相關系所之學生、授課教師、現有 EDA 工具與開源社群	幫助學生快速理解複雜開源專案並產出高品質專題報告，同時提升教師評估程式碼架構的效率。為開源社群提供強大文件輔助工具，並有機會促使傳統 EDA 廠商投入 AI 製圖的創新。

（二）問題解決策略

1. 團隊預計提出的解方要點與願景？

※備註：建議在論述過程中說明目前類似問題的常見解方為何？團隊預計提出的解方又有何創見？

目前對於製作簡明扼要的三種描述圖，有以下幾種作法，本團隊認為皆有其施行上的限制。

首先，便是最傳統的手繪，使用draw.io或類似的圖表工具，手動繪製並標記每個模組的接線與名稱，對於核心開發人員繪製本身仍然會是一個相當程度的繁瑣工作，因為要讓圖表用易於他人理解的方式呈現本來就相當困難。

再者，則是用EDA公司提供的工具，這類工具固然經過嚴格的驗證流程絕對準確，但缺乏篩選重要性訊號的能力，由於硬體接線錯綜複雜，再未經合成前的硬體描述語言有無數種寫法可以對應到相同的抽象圖，不過這類工具會完全忠實地把所有訊號線一次列出來，不僅非常繁瑣，在足夠複雜的系統中幾乎是完全無法直接閱讀或寫進文件。

最後，則是直接將撰寫好的硬體描述語言丟給 LLM，在極為簡單易理解的程式裡這是一個可行的方法，但在稍大的系統中，就可能出現錯誤接線、幻想接線、多接、少接、不僅僅是造成多餘的修改時間，也隱含了這個方式的不可驗證性，對於追求正確性的硬體工作，這是非常致命的一個問題，也是本計畫的核心解決目標。

我們的解法預計結合LLM及AI的處理抽象層級的優勢，以及演算法、邏輯驗證的準確性來打造一個從RTL到圖表的流水線，初步預計分成三步執行。

1. 由 LLM 閱讀 RTL，篩選出 AI 覺得重要的訊號列成一張清單，覺得需要模組化的列成第二張清單。
2. 透過演算法 trace 重要訊號的相關訊號，確定從輸入、輸出、重要訊號間的邏輯是自洽的，不是的話在這步補上可滿足邏輯描述的訊號。模組的部分確保單一模組的輸入輸出可以被正確包覆沒有遺漏。
3. 第三步是生成與驗證，只要確定各模組單元、接線、即可針對給定圖表大小做一次排版的演算法，這樣圖表的位置擺放與接線流暢度可以被最佳化，並在這步可以加跑一次邏輯驗證確定整個系統的邏輯自洽。

2. 上述解方的具體實施步驟與過程記錄方式？

為確保解方能兼顧 **LLM 的高階抽象化能力**與**傳統演算法的絕對準確性**，本專案將採取模組化與敏捷式開發，具體實施步驟分為三階段：

- 第一階段：語意萃取與抽象化模組開發（九月）
    - 步驟：建立 RTL（以 Verilog/SystemVerilog 為主）的測試資料集，設計 System Prompt 引導 LLM 閱讀原始碼，要求其輸出包含關鍵控制訊號與資料流的清單（例如 JSON 格式）。
    - 測試點：確認 LLM 是否能正確辨識如跨模組握手訊號（Handshake）、狀態機核心暫存器等關鍵特徵。
- 第二階段：邏輯演算法追蹤與自洽性驗證模組開發（十月）
    - 步驟：開發基於抽象語法樹（AST）的硬體訊號追蹤演算法。將第一階段 LLM 挑選出的「重要訊號清單」輸入此演算法，透過圖論（Graph Theory）追蹤輸入端至輸出端的路徑。若發現斷點或邏輯不連續，則由演算法自動補齊必要的隱藏訊號，確保模組的封閉性與自洽性。

- 第三階段:Agent 整合、排版最佳化與圖表生成(十一月～十二月)
    - 步驟: 將上述兩模組與繪圖引擎(如 Graphviz、Mermaid 或開源排版演算法)整合。開發排版最佳化功能, 針對模組大小、接線交錯最少化進行計算, 最終輸出易讀的硬體架構圖、狀態機圖與時序圖。

過程記錄方式將透過 Git 進行程式碼版本控制與團員協作, 再來也會建立 Notion 或 HackMD 共享專案頁面來進行會議和開發日誌記錄。

3.「代理式AI」(AI Agent)於上述解方中扮演何種關鍵角色？

在本解方中, AI Agent 並非單純的程式碼產生器或對話機器人, 而是扮演架構和驗證居中協調的角色。

因先進半導體製成時程, 硬體設計對正確性有著零容忍的標準, 純 LLM 容易產生幻覺接線。因此, 我們設計的 AI Agent 會配備外部工具, 形成一個「生成 ➔ 驗證 ➔ 修正」的閉環系統:

1. **認知與規劃:** Agent 讀取龐大的 RTL 專案後, 先規劃出系統的層次結構。
2. **呼叫工具:** Agent 主動呼叫團隊開發的「邏輯自洽性演算法」來驗證自己初步整理的架構。
3. **自我反思與修正:** 若演算法回報「訊號 A 與模組 B 之間缺乏實體連線路徑」, Agent 能夠根據回報的錯誤資訊, 重新檢視 RTL 程式碼, 修正自身的理解並重新輸出架構描述。透過這種代理式的運作邏輯, AI Agent 不僅是製圖者, 更是具備驗證能力的架構審查員, 從根本上解決傳統 LLM 無法保證硬體邏輯絕對正確的擔憂。

(三)、專案成果

1. 專案執行後預計取得哪些成果？

- **核心軟體工具:** 一套可自動化執行的 RTL-to-Diagram 命令列工具(CLI)或網頁端原型。使用者只需輸入 Verilog/SystemVerilog 專案資料夾, 即可一鍵生成經過驗證的硬體架構圖(BlockDiagram)、狀態機圖(FSM)與時序圖(Timing Diagram)。
- **壓力測試案例集:** 以真實世界中具備一定複雜度的開源專案(例如開源 RISC-V 核心), 以及高度重視資料流與模組化設計的硬體感知機器學習加速器作為基準測試, 展示本工具在處理極端複雜架構時, 能夠相較於傳統 EDA 工具有更高的「易讀性」。

- **開源技術文件**:將我們的成果在 Github 上開源並撰寫詳細的 Readme，可供學術界或開源硬體社群延伸開發。

2. 團隊將透過哪些評鑑方法來確認成果達標？

- **邏輯準確度— 目標:100% 零幻覺** 生成的圖表中，所有呈現出來的模組、接線與狀態轉換，必須在原始 RTL 程式碼中 100% 找得到實體對應，絕不允許出現憑空捏造的幻覺連線。這將透過團隊開發的圖論追蹤演算法進行自動化驗證。
- **抽象化降噪率— 目標:減少 60% 以上的冗餘節點** 定義「降噪率」為:(傳統 EDA 工具產生的完整訊號線與節點總數 - 本工具產生的節點與連線數) / 傳統 EDA 總數。目標是在「不影響頂層控制與資料流理解」的前提下，替使用者過濾掉 60% 以上無助於人類理解的底層微觀連線。
- **易讀性與時間成本測試** 邀請電機系大學部具備數位 IC 基礎的學生進行 User Study（盲測）。給予一份陌生的中型 RTL 程式碼，A 組給予傳統原始碼與 EDA 展開圖，B 組給予本工具生成的圖表，測量兩組受試者回答出「核心資料流路徑」與「狀態機轉移條件」所需的平均時間，目標為減少 50% 的理解時間。

3. 專案成果預計產生哪些影響力（impact）？

- **學術與教育影響**: 對於正在修習數位 IC 設計、計算機結構的學生而言，這套工具能大幅降低閱讀複雜開源硬體專案（如 RISC-V）的門檻，幫助他們專注於「架構設計思維」而非迷失在程式碼中。
- **業界開發流程升級**: 解決業界長期以來「程式碼更新，但規格文件未更新」的痛點。作為文件自動化生成工具，它能輕易整合進硬體開發的 CI/CD 流水線中，提升研發團隊溝通與傳承的效率。
- **跨領域創新**: 證明了 LLM 不僅能拿來「寫」Code，透過 Agent 架構與傳統確定性演算法（Deterministic Algorithms）的互補，更能作為高精密工程領域（如半導體設計）中，處理「認知與抽象化任務」的強大輔助大腦，且可以參與邏輯驗證這種步容許錯誤的任務。

三、預算規劃

※備註:請依專案實際執行情形進行評估與規劃，不受限於本計畫可核銷項目，若有需要可自行製表以利說明。

概述	單價	數量	費用估計（台幣）
API 費用	3000	無	3000
Claude Code	3000	4個月	12000

【注意事項】:

1. 計畫書總頁數不可超過10頁，檔案大小以10MB為限，若有圖表或其他補充資料，可另行製作附件一併繳交。
2. 計畫書請於2026/09/20（日）前，以PDF檔形式寄至ntulsctld@ntu.edu.tw，信件主旨為：115-1 NTU Challengers專案申請－團隊主要聯繫人姓名。
3. 計畫書寄送完成後，報名表單也須於期限內填寫完畢，才算完成報名手續，缺件者恕不受理。